

第二章 操作系统概述

- 1. 操作系统概念，与一般软件的区别

1. 它是计算机系统中最基本的**系统软件**（③**是最接近硬件的一层软件**）
2. 操作系统是指控制和管理整个计算机系统的硬件和软件资源，并合理地组织调度计算机的工作和资源的分配（①**操作系统是系统资源的管理者**）
3. 以提供给用户和其他软件方便的接口和环境（②**向上层提供方便易用的服务**）
4. 区别：
 1. 与普通计算机软件作用相同，是处理机执行的一段程序
 2. （意图）操作系统控制处理器及其他系统资源，并控制其他程序的执行时间；会释放控制，而且必须依赖处理机才能恢复控制

- 2. 操作系统的功能和目标

功能

1. 处理机管理、进程管理、内存管理、文件管理、设备驱动管理（如摄像头等资源）、用户接口
2. 管理计算机的软硬件资源
3. 提供各种接口
4. 对硬件机器的扩展

目标

1. 方便：操作系统使计算机易于使用
2. 有效：操作系统允许以更有效的方式使用计算机系统资源、
3. 扩展能力：在构造操作系统时，应允许在不妨碍服务的前提下，有效地开发、测试和引入新的系统功能

- 3. 操作系统的构成模块

对应上题中要实现的功能有相应的模块

- 4. 操作系统发展的三主线

1. 历史发展角度
 1. 串行处理：手工，直接与计算机硬件打交道（无OS）
 2. 批处理

- 单道：把作业按顺序组织策划成批，并将整个批作业放入输入设备，等待程序执行结果
- 多道（OS开始出现）
- 3. 分时（以时间片为单位轮流为各个用户/作业服务，可以**人机交互**）
- 4. 实时（软实时、硬实时）

2. 设计实现角度

3. 其他

- 5. 分时系统与批处理系统（多道）的概念和异同

1. 分时系统：计算机为每个用户提供一定时间片的CPU时间，以使用户可以执行他们的任务并与计算机交互
2. 批处理系统：用户提交一个作业，该作业包含一系列需要执行的任务，然后计算机将按照顺序逐一执行这些任务。
3. 分时：用户通过终端与计算机交互 批处理：用户使用作业控制语言命令和作业提供的命令
4. 分时：减小响应时间 批处理：充分利用处理器
5. 都使用了多道程序设计

- 6. 中断和异常在分时系统和批处理系统中的差异

1. 在分时系统中，异常处理和中断处理非常重要。当一个程序出现异常时，操作系统会立即停止当前进程并将控制权转移到异常处理程序。
2. 在批处理系统中，通常只有一个用户提交作业，如果一个作业出现错误，系统可能会记录错误信息并将作业标记为失败，而不是立即停止当前作业并转移到异常处理程序。

- 7. 如何理解操作系统与虚拟机

虚拟机是指通过软件模拟的具有完整硬件系统功能的、运行在一个完全隔离环境中的完整计算机系统

操作系统是一个虚拟机，（CPU被虚拟，并发变并行，内存被虚拟化）

第三章 进程

- 1. 为了实现进程，需要CPU提供哪些硬件支持（mode态，中断与异常）

当进程需要进行输入输出、系统调用或遇到错误时，CPU需要提供中断/异常处理机制来实现从用户态到内核态的切换，并执行相应的中断/异常处理程序

1. 中断控制器：用于处理硬件中断和软件中断，以便CPU能够及时响应和处理系统中的事件
2. 内存管理单元：用于实现虚拟内存，将虚拟地址映射到物理地址，以便进程能够访问到自己的内存空间
3. 寄存器：用于存储CPU执行指令时所需要的数据和地址，如通用寄存器、程序状态字寄存器等
4. 程序计数器：用于存储当前执行指令的地址，以便在下一次执行时能够继续执行
5. 栈指针：用于存储程序执行过程中的局部变量、函数参数和返回地址等信息
6. 计时器：用于计算程序执行时间，以便操作系统能够进行时间片轮转调度，实现进程切换

- 2. 进程的概念，产生背景，和程序的关系和区别

- 程序代码 + 代码相关联的数据集 + PCB；
- 进程：动态的，一个正在执行的程序实例，具有自己的内存空间、寄存器、程序计数器、栈等资源
- 程序：静态的，就是一组指令和数据的集合，程序只有被加载到内存中被执行时才会变成一个进程
- 一个程序可以对应多个进程，即同一个程序可以被多个用户同时执行，每个用户都有自己的进程实例
- 背景：如果程序是直接加载到内存中执行的，如果一个程序出现错误，可能会导致整个系统崩溃；

可以让os更方便地管理资源，实现资源对多个程序可用；

- 3. 进程控制块的概念及功能，简单描述其构成

- 有操作系统创建并管理；存储基本的进程描述信息的数据结构；操作系统对进程进行管理所需的信息都存在PCB中
- 可以中断一个程序的执行，并在后来恢复程序的执行
- 回答三个字段：eg. ID（唯一标识符）、状态（运行态等）、优先级

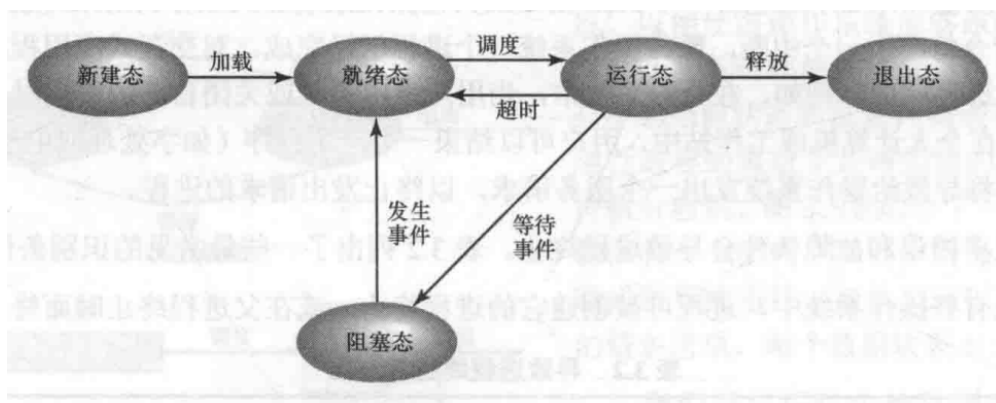
- 4. 从程序生命周期角度说明一个程序是如何转化为一个进程的 (必须画出PCB)

1. 程序加载：一个程序通过**编译链接**后得到的可执行文件加载到内存中
2. 进程创建：分配内存空间，创建PCB
3. 进程初始化：将代码、数据、**堆**、**栈**加载进内存空间

- 5. 进程控制相关的原理或者系统调用 进程创建销毁等 (fork wait)

- 进程控制：实现进程状态转换；mode的切换；中断
- `fork()` 函数产生一个子进程，子进程返回0，父进程返回子进程ID，出错返回-1
- `wait()` 函数会阻塞当前进程，直到子进程结束
 1. 在进程表中为新进程分配一个空项
 2. 为子进程分配一个唯一进程标识符
 3. 复制父进程的进程映像，但共享内存除外
 4. 增加父进程文件的计数器，反映另一个进程现在也拥有这些文件的事实
 5. 将子进程设置为就绪态
 6. 将子进程的ID号返回给父进程，将0值返回给子进程

- 6. 进程的五状态图



- 7. 从三个角度来分析进程交替执行

1. OS：让程序 A 执行6个指令周期，当满足进程切换的条件时立刻停止 A 的继续执行，收回控制权并执行分派器的程序，将控制权转给程序B
2. CPU：CPU 正在读取A程序的指令并执行，然后去读取B进程的指令并执行（意识不到进程的存在）
3. 某个进程：意识不到自己被中断，虽然在物理控制流上是非连续的，但在逻辑控制流上是连续的

- 8. 进程的创建流程，并结合Linux的fork系统调用流程

1. 为新进程分配进程标识符
2. 为进程分配空间
3. 初始化进程控制块
4. 设置正确的链接（放在对应的调度队列）
5. 创建或扩充其他数据结构
6. `fork` 的一次调用两次返回（2'）

- 9. 关于进程切换的流程和时机（以Linux 的wait为例）

流程

1. 模式切换：用户态->内核态
2. 保存当前进程的上下文，更新PCB（状态等字段）并将进程移到相应的队列
3. 选择下一个进程并恢复其上下文信息，更新PCB

时机：只要产生中断或异常就可能会有进程切换态的切换（中断发生后不一定切换进程）

（考研）给代码解释

线程

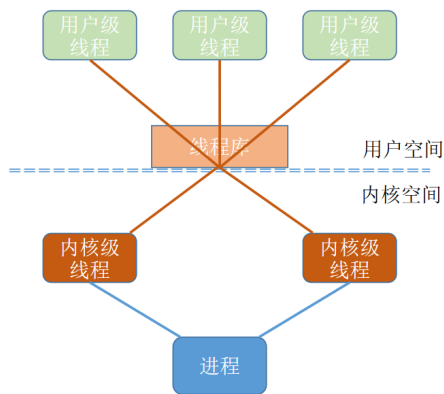
第四章

- 1. 线程的概念，进程和线程的区别（关系）

- 线程是调度的基本单位，是一个基本的CPU执行单元，也是程序执行流的最小单位（轻量版的进程）
- 进程是资源分配的基本单位，线程是调度的基本单位；一个进程内的线程共享进程的资源

- 2. 线程的实现方法

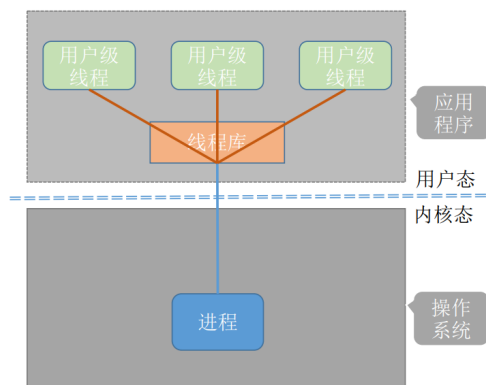
1. 用户级线程：所有的线程管理工作都由应用程序负责，在用户态下完成，操作系统内核意识不到线程的存在，任何应用程序都可以使用线程库设计成多线程程序
2. 内核级线程：线程的管理工作由操作系统内核完成（通过TCB管理），在内核态下完成，操作系统内核可以意识到
3. 用户级+内核级：



3. ULT和KLT各自的优缺点

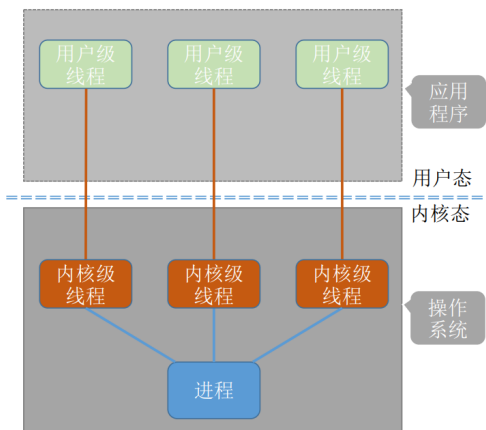
1. ULT

- 优：用户级线程的切换在用户空间即可完成，不需要切换到内核模式，节省了两次状态转换的开销
- 调度因应用程序的不同而不同
- 缺：当一个用户级线程被阻塞后，整个进程都会被阻塞，并发度不高。多个线程不可在多核处理机上并行运行。



2. KLT

- 优：当一个线程被阻塞后，别的线程还可以继续执行，并发能力强。多个线程可在多核处理机上并行执行
- 缺：一个用户进程会占用多个内核级线程，线程切换由操作系统内核完成，线程管理的成本高，开销大



3. 用户级线程是“代码逻辑”的载体，内核级线程是“运行机会”的载体，一段“代码逻辑”只有获得了“运行机会”才能被 CPU 执行；内核级线程才是处理机分配的单位

第五章 并发性：互斥与同步

- 1. 并发需要解决的四个问题，及概念

1. 同步：进程的并发必然导致异步性，而实际中有些程序必须按照顺序执行（如：读&写），这种直接制约关系就是同步
2. 互斥：当一个进程进入临界区访问共享资源时其他进程不能进入该临界区访问任何共享资源的情形
3. 饥饿：一个可运行的进程即使能继续执行，但被调度程序无限期地忽视，而不能被调度执行的情形
4. 死锁：一组进程中的每个进程都在等待某个事件，而仅有这组进程中被阻塞的其他才可触发该进程时，就称这组进程发生了死锁

- 2. 同步和互斥的产生背景（为什么引入）

现代操作系统基本都是多任务操作系统，即同时有大量可调度实体在运行

都需要访问/使用同一种资源

进程的并发必然导致异步性，而实际中有些程序必须按照顺序执行（如：读&写），这种直接制约关系就是同步；会产生**条件竞争**，保护数据；

- 3. （考研特色题）什么叫条件竞争，并举例描述（结合代码）

多个进程或线程读写数据时，最终结果取决于多个进程的指令执行顺序

进程 P1	进程 P2
•	•
chin = getchar();	•
•	chin = getchar();
chout = chin;	chout = chin;
putchar(chout);	•
•	putchar(chout);
•	•

- 4. 为了解决同步和互斥问题，有哪些解决方案

1. 软件

- dekker
- Peterson

2. 硬件：中断禁用；专用机器指令（`compare_and_swap` & `exchange`）

3. 系统：管程&信号量&消息传递

- 5. 如果用硬件解决互斥方法，他有什么缺点

1. 关中断：简单粗暴，操作系统被限制只能交替运行程序，执行的效率降低；不能用于多处理器体系结构，只会屏蔽当前运行的处理机，其他处理器仍然可能对其中断
2. 特殊的硬件指令：不满足“忙则等待”；选择等待的进程是任意的，可能会饥饿；可能会死锁：低优先级临界区被中断，高优先级忙等待，由于被中断的优先级低，不会再被调度

- 6. 信号量的概念本质及其实现

大题

- 7. 管程的好处或引入管程的原因是什么

管程——>函数库级的信号量，属于系统库，所有同步机制限制在管程内部

信号量分布在整个程序中，使用不当，可能造成程序错误，甚至死锁。管程易于验证同步正确性，易于检测出错误

让程序员写程序时不需要再关注复杂的PV操作，让写代码更轻松

第六章 并发：死锁和饥饿

- 1. 死锁的概念和产生背景

给一段代码判断是否是死锁（过独木桥，过十字路口）

一组进程中的每个进程都在等待某个事件，而仅有这组进程中被阻塞的其他才可触发该进程时，就称这组进程发生了死锁

背景

1. 竞争有限的资源（可重用资源、可消耗资源：都会造成死锁）
2. 进程推进顺序不当
3. 资源分配不当
4. 进程运行推进阻塞

- 2. 死锁的条件（三个充分，四个充要）

1. 互斥：一次只能有一个进程使用一个资源，其他进程不能访问已经被分配给其他进程的资源
2. 占有且等待：一个进程等待其他资源时，继续占有已分配的资源
3. 非抢占：只能等其他进程主动释放自己的资源，不能去抢占其他进程的资源
4. 循环等待：存在一个闭合的进程链，每个进程至少占有此链中下一个进程所需要的资源（必要不充分；发生循环等待未必死锁）

- 3. 死锁的解决方法

1. 预防：破坏四个条件之一

1. 互斥：不能被破坏
2. 占有且等待：一次性请求所需要的所有资源（依然还是有占有且等待条件的）
3. 非抢占：允许抢占（破坏了抢占的条件）；申请资源被拒绝就释放自己的全部资源（不占有）
4. 循环等待：定义资源类型的线性顺序（破坏了循环等待的条件）

2. 避免（银行家算法）

1. 一个进程请求会死锁就不启动该进程
2. 一个进程的资源请求会死锁，就不分配该资源

3. 检测（死锁了怎么办，挂起谁？）

1. 银行家算法检测

2. 恢复

1. 取消所有死锁进程
2. 回滚进程到检查点并重新启动
3. 连续取消死锁进程直到不再死锁
4. 连续抢占资源直到不再死锁

4. 综合：把资源分成几组不同的资源类，使用线性排序策略，在一个资源类中使用最适合该资源的算法

- 4. 死锁和饥饿的关系

都是竞争资源引起的

死锁是导致饥饿的一个原因（不是唯一）

死锁等待对方释放资源（至少两个进程）

饥饿自己拿不到资源，不一定是对方的（一个进程也可饥饿）

第七章 内存管理

- 1. 简述存储管理的七个方法

1. 全部加载且连续：

- 固定分区：内存被划分成大小固定的静态分区，进程可以装入大于等于自身大小的分区中
- 动态分区：分区是动态创建的，进程可以放入与自身大小相等的分区

2. 全部加载不连续：

- 简单分页：内存被划分成页框，进程被划分成等大的页，把进程包含的所有页都装入内存内不一定连续的某些页框中
- 简单分段：进程根据不同的功能划分成不同的段，相应地进程也被划分为大小不等的段，进程包含的所有段都装入内存内不一定连续的某些动态分区中

3. 部分加载不连续：

- 虚拟分页：不需要装入一个进程的所有页，在需要时自动调入
- 虚拟分段：不需要装入一个进程的所有段，在需要时自动调入

- 2. 固定分区、动态分区、分页、分段的异同（+是否全部加载、是否连续）

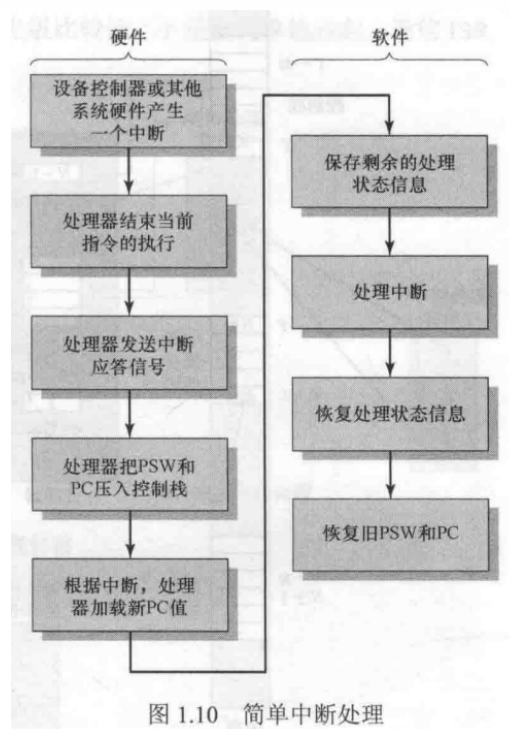
分页、分段：一个大小相同，一个大小不同

固定分区、分页：大小相等

动态分区和分段：大小不等

- 3. 段表和页表的构造和使用

是由谁构造（操作系统内核）、什么时候构造（进程创建时分配连续的虚拟空间并构造对应的页表和段表）、构造出来长什么样子（表格：页号、页框号；段号、长度、基地址）；谁在使用（CPU）、什么时候使用（进程访问时）



第八章 虚拟内存

1. 实现虚拟存储主要考虑的三个问题

1. 是否支持虚拟存储
2. 虚拟存储的具体实现方式（分页、分段、段页）
3. 操作系统在内存分配时的具体算法（os考虑）

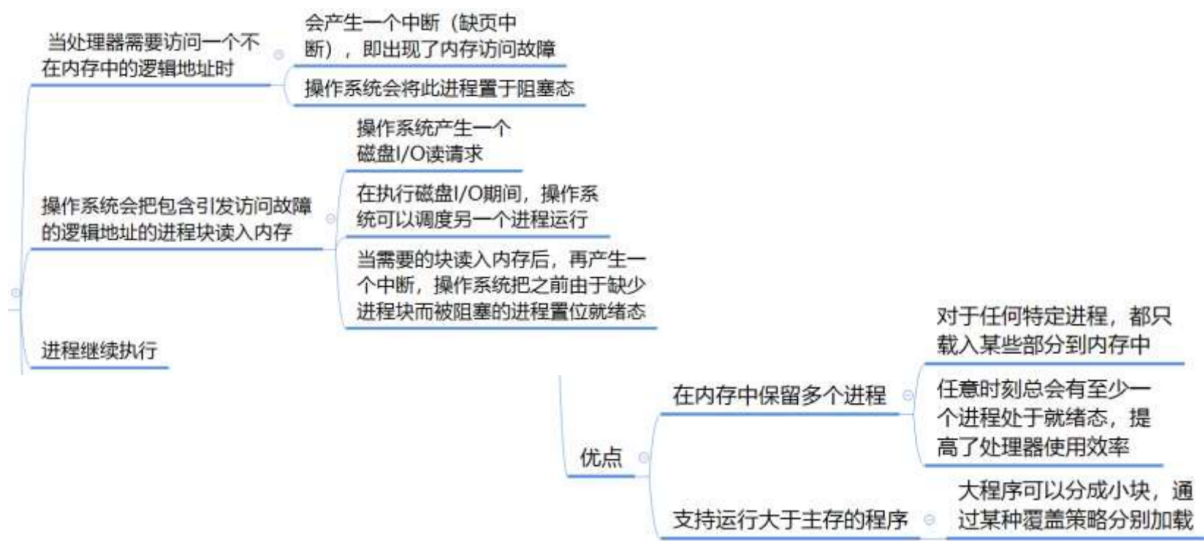
2. 实现虚拟存储的三个方法

1. 虚拟分页：不需要装入一个进程的所有页，在需要时自动调入
2. 虚拟分段：不需要装入一个进程的所有段，在需要时自动调入
3. 先分段再分页

在段页式系统中，用户的地址空间被程序员划分为许多段。每段依次划分为许多固定大小的页，页的长度等于内存中的页框大小。若某段的长度小于一页，则该段只占据一页。从程序员的角度看，逻辑地址仍然由段号和段偏移量组成；从系统的角度看，段偏移量可视为指定段中的一个页号和页偏移量。

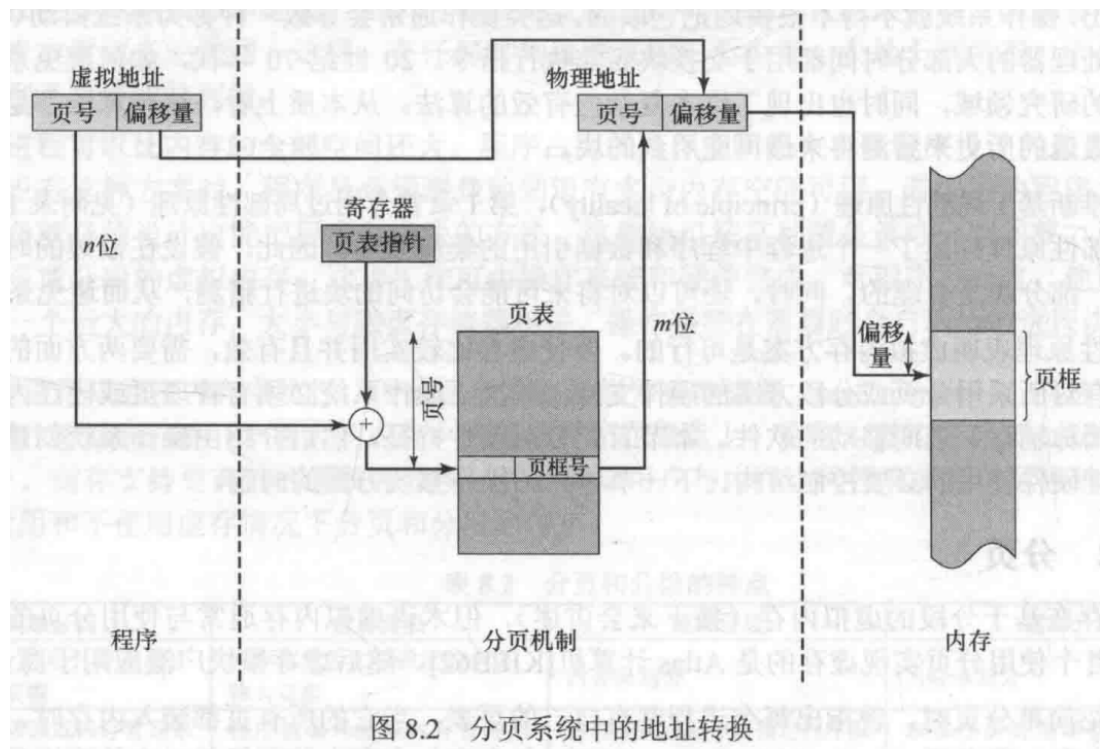
3. 虚拟存储器的PageFault的处理流程

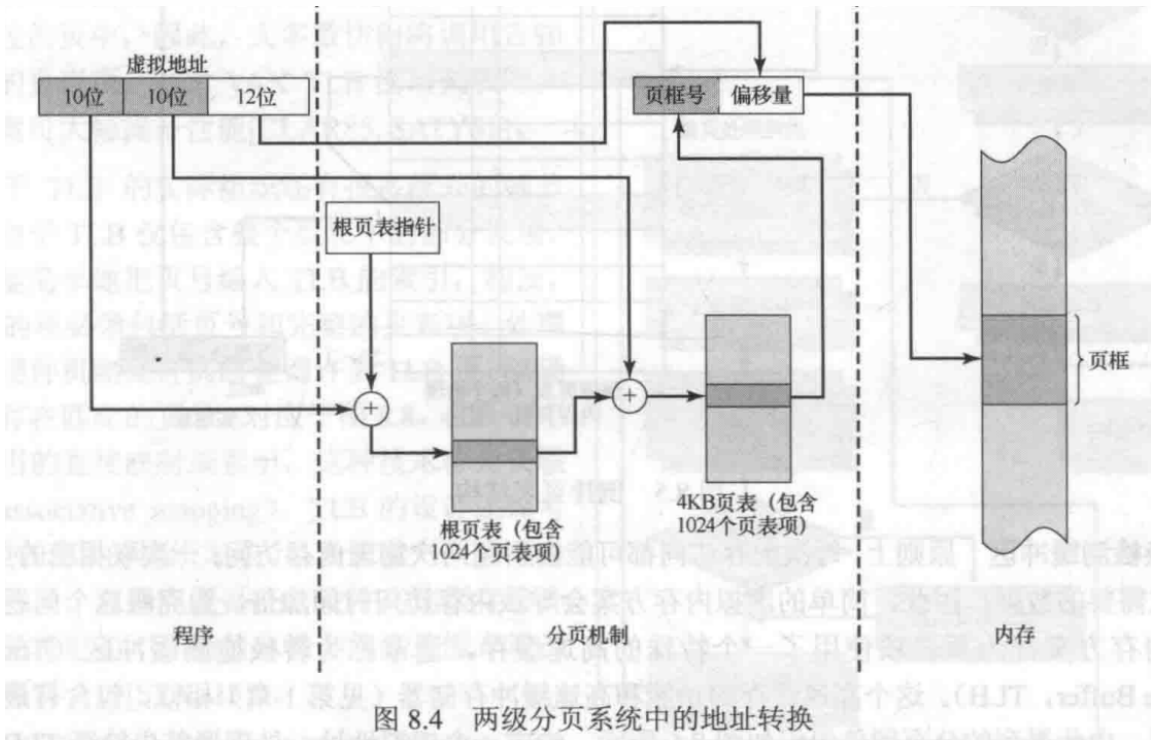
执行流程



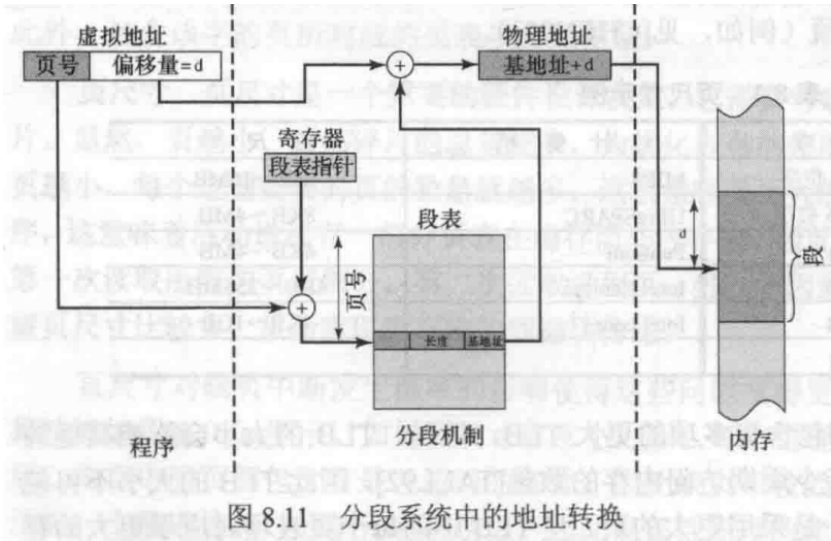
4. 地址转化（只考图）

1. 虚存分页（两级页表）怎么查

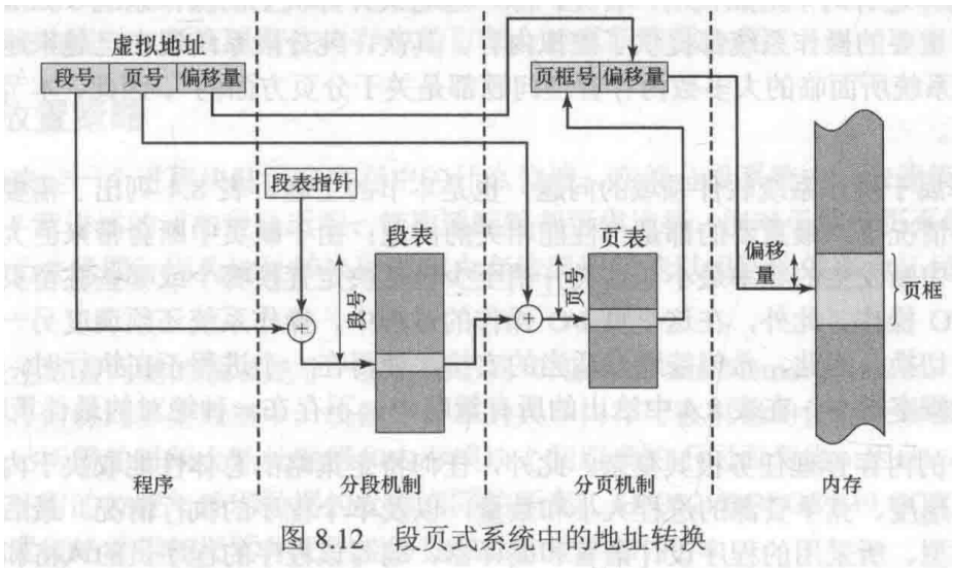




2. 虚存分段怎么查

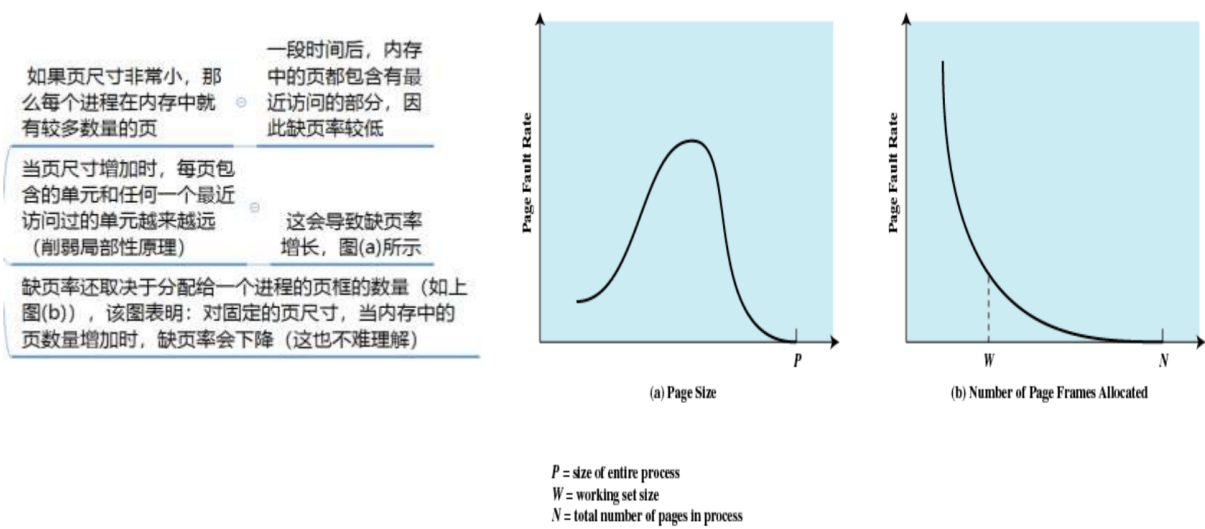


3. 虚存分段+分页怎么查



5. 请简述缺页率和页尺寸和页框数的关系（结合图分析）

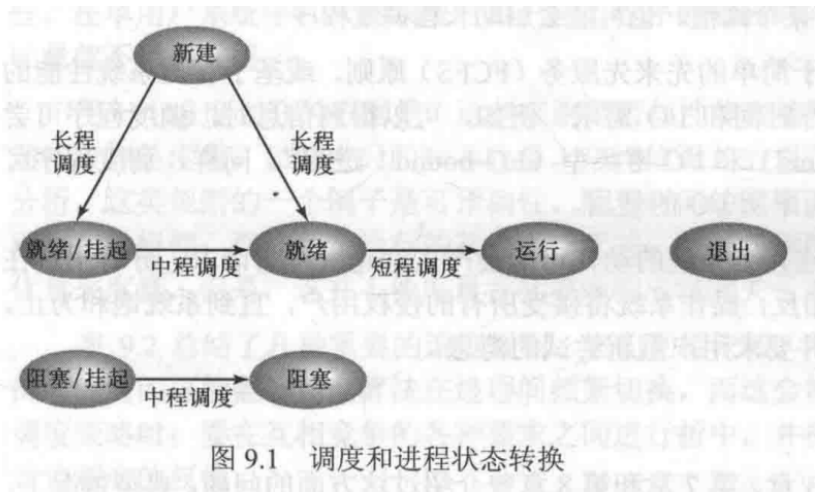
缺页率和页尺寸之间的关系



第九章 单处理器调度

1. 长中短三个调度的描述（三个图：状态转换、调度层次、队列管理）

- 1. 长程调度（作业调度）：按照某种规则，从后备队列中选择合适的作业将其调入内存，并为其创建进程（无->创建态->就绪态）（外存->内存）
- 2. 中程调度（内存调度）：按照某种规则，从堵塞挂起或就绪挂起队列中选择合适的进程将其数据调回内存（挂起态->就绪态（阻塞挂起->阻塞态））（外存->内存）
- 3. 短程调度（进程调度）：按照某种规则，从就绪队列中选择一个进程为其分配处理机（就绪态->运行态）（内存->CPU）



- 1. IOBuffer的产生背景和常见的IOBuffer的实现方式

1. 产生背景
1. 当I/O速度较慢时会阻塞进程（减少了进程被阻塞的时间）

2. IO操作限制了系统调度，正在使用IO设备的进程不能被换出（进程可以被安全地换出）

3. 进程发出IO请求后被换出，等待IO发生，IO等待进程被换入，发生死锁

4. 减少对CPU的中断频率，放宽对中断响应时间的限制

5. 提高CPU和I/O设备之间的并行
2. 实现方式
1. 单buffer：只有一个缓冲区，数据的读写都在同一个缓冲区中进行；进程读取数据后立刻请求下一个数据块（预读）

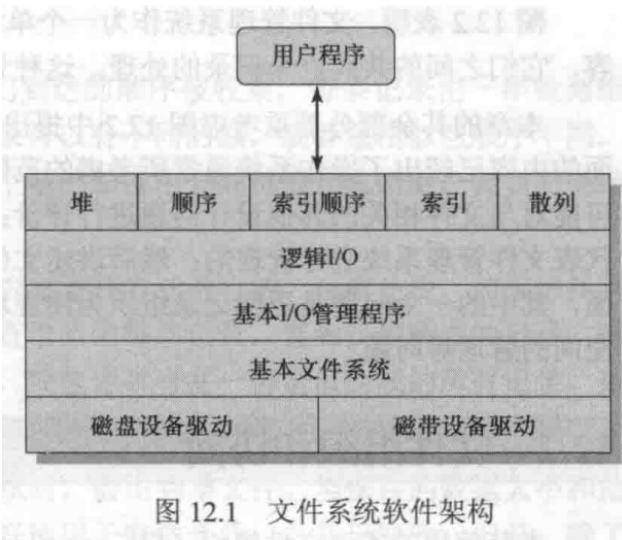
2. 双buffer：有两个缓冲区，数据的读写可以同时进行，互不干扰；

3. 循环buffer：有多个缓冲区，构成了一个循环使用的缓冲区序列

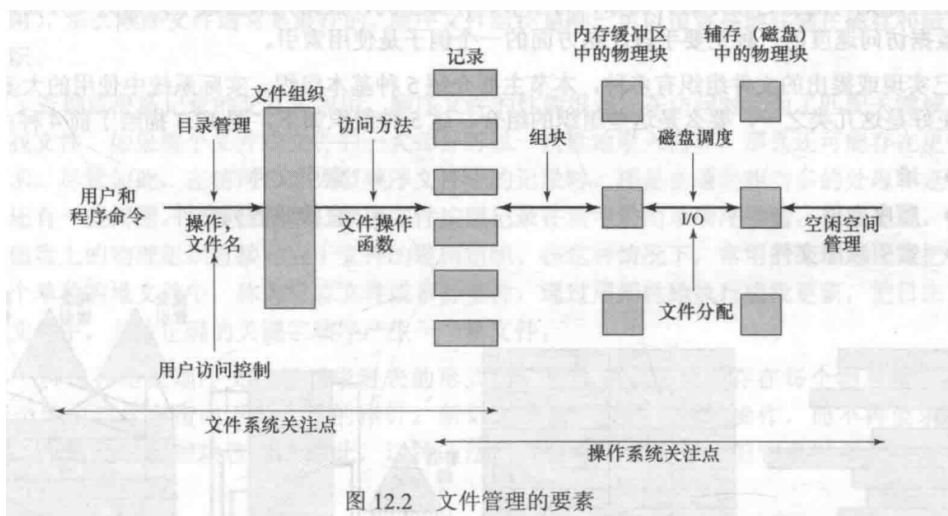
第十二章

文件管理

- 1. 文件系统的设计层次（图）



1. 设备驱动：直接与外围设备通信（os）
2. 基本文件系统：与计算机外部环境的基本接口（os）
3. 基本IO管理程序：负责所有文件IO的初始化和终止（os）
4. 逻辑IO：维护关于文件的基本数据，使能够访问记录



用户和应用程序执行文件操作在记录级上执行，将文件视为具有组织记录的某种结构，但IO是以块为基础的，将块组织成记录

- 2. 文件系统逻辑层次上的五种实现方式（堆、索引）

1. 堆：按照数据到来顺序进行组织（记录和记录中的域都是变长的；穷尽搜索）
2. 顺序文件：文件由定长的记录构成，每个记录由确定顺序（关键域唯一标识）的、定长的域构成（穷尽、排序存储）
3. 索引顺序文件：只为一个关键域建立索引，比顺序文件多了支持随机访问的文件索引和溢出文件
4. 索引文件：可以为多个关键域建立索引
5. 散列文件：采用散列表索引，不按照关键域排序（有关键域；域和记录等长；快速访问）

- 3. 文件系统物理层次上的三种实现方式

1. 连续分配：创建时分配一组连续的块，每个文件只需要一个表项用于说明起始块和文件的长度，会出现外部碎片，需要执行紧缩算法来释放磁盘中的额外空间
2. 链式分配：链式分配基于单个块，链中的每块都包含指向下一块的指针，要一个表项用于说明起始块和文件的长度，没有外部碎片，最适合顺序文件，局部性原理不再适用，需要周期性地合并文件
3. 索引分配：每个文件在文件分配表中都有一个一级索引，分配给该文件的每个分区在索引中都有一个表项